



# Fundamentos da Programação Orientada a Objetos

# Encapsulamento

Imagine, por exemplo, que você tenha uma conta bancária. Eu suponho que você não gostaria que qualquer pessoa pudesse acessar (e modificar) o saldo de sua conta diretamente, certo? Em vez disso, eu acho que você gostaria de que o seu saldo fosse acessado por meio de um conjunto específico de operações que garantam que ele não seja manipulado de maneira inadequada. Algumas operações poderiam ser depósitos, retiradas, Pix etc. O encapsulamento ajuda a garantir isso em seu código.

Quase todas as empresas modernas usam alguma forma de programação orientada a objetos, portanto, **quase todas as empresas usam encapsulamento em seu código**. As empresas valorizam (e testam nas entrevistas de emprego) desenvolvedores que saibam escrever códigos que sejam seguros, fáceis de manter e flexíveis. Por isso, se você sabe como criar códigos com encapsulamento, também saberá produzir um código de alta qualidade e, como você deve imaginar, quem escreve códigos com alta qualidade possui maiores chances de ter acesso a bons salários.

Então, nesta Unidade, criaremos algoritmos com **encapsulamento**: programas que ocultam os detalhes de implementação de uma classe, impedindo o acesso direto ao estado do objeto (atributos). Nesse caso, o acesso aos atributos ocultos pode ser feito apenas por meio de sua interface (métodos públicos).

Vamos recordar: abstrair é observar um cenário real (como uma empresa, um processo, um serviço ou um material) com o objetivo de isolar um elemento, ou um conceito, com o qual precisamos trabalhar, excluindo os demais.

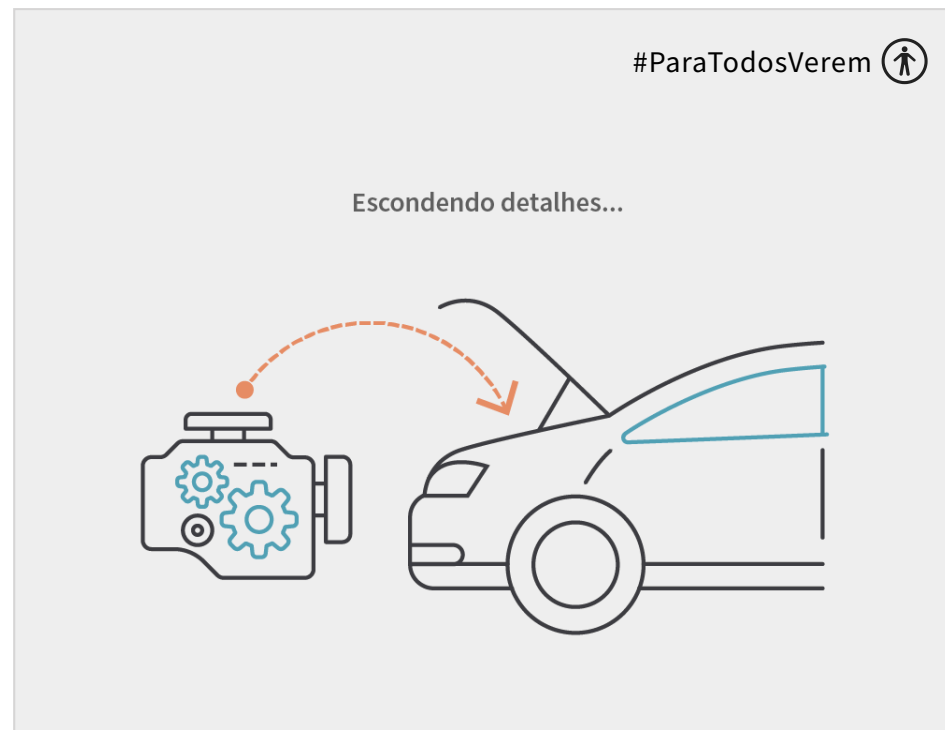
Usamos um processo de abstração para realizar o encapsulamento das classes, que consiste em separar a visão do criador dos objetos da visão do usuário desses objetos. Para isso, o criador do objeto, ou programador, “esconde” a parte interna da classe (detalhes de lógica e atributos) da parte externa da classe, que é acessada pelos usuários dos objetos da classe. A visão externa da classe é chamada de interface da classe.

Para entender melhor este conceito vale a pena ver a figura a seguir. Uma analogia que costumamos usar para explicar encapsulamento é o conceito de um carro: a maioria dos motoristas não precisam saber como o motor, a suspensão e os freios foram construídos – os motoristas estão apenas interessados em usar um carro para se deslocar de um ponto a outro. Os motoristas não precisam acessar todos os cabos, peças e

soldas do carro, mas somente precisam dos pedais, do volante e do câmbio na maioria das vezes. Os pedais, volante e câmbio são esta “visão externa da classe” que comentamos anteriormente.

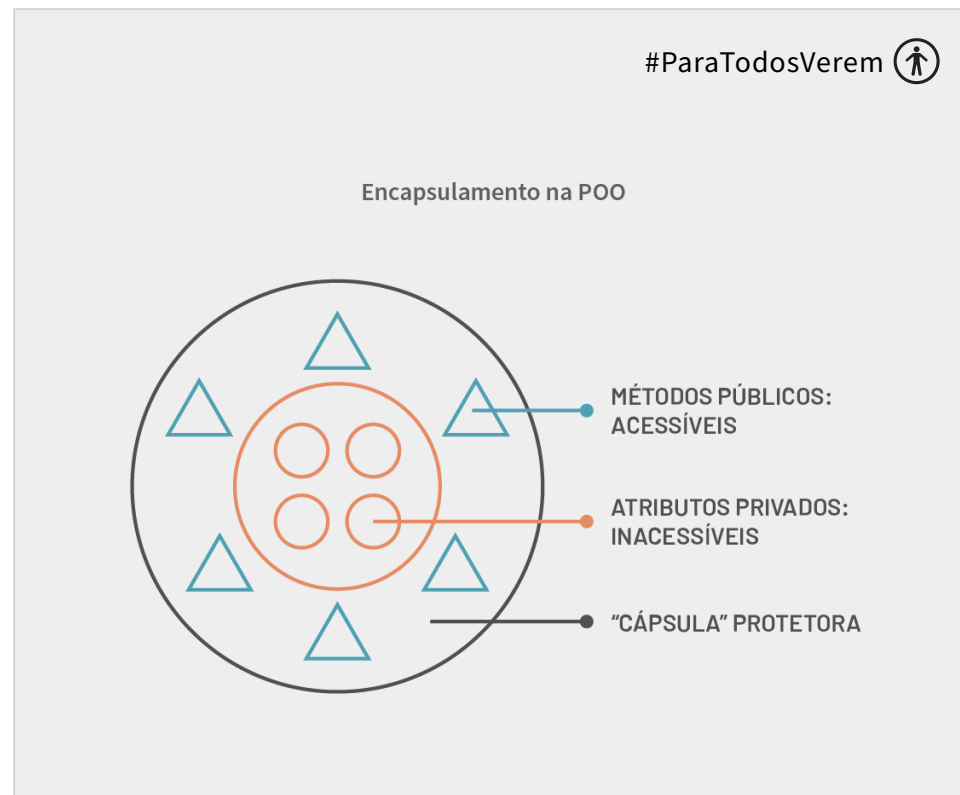
## + Encapsulamento (conceito geral)

Para utilizar o carro, o motorista não precisa conhecer como o motor é feito e nem como ele funciona. Logo, esses detalhes ficam **ocultos** em uma cápsula, ou encapsulados, para o usuário do carro, que é o motorista.



## + Encapsulamento na POO

Da mesma forma, os usuários dos objetos de uma classe (que podem ser objetos de outras classes ou outras aplicações) não precisam conhecer todos os detalhes de implantação da classe em questão: basta conhecer a **interface** (que são seus métodos públicos). Logo, os detalhes de implantação da classe também são **ocultos**, ou encapsulados.





### SAIBA MAIS

Como a obra *Java Básico*, sobre a qual o professor comenta na videoaula, não está mais disponível em nossas bibliotecas virtuais, separamos alguns materiais que são igualmente úteis nos seus estudos:

[Java 101: como começar a aprender Java](#)

SCHILDT, Herbert.  [Java para Iniciantes](#). 6. ed. Porto Alegre: Bookman, 2015.

## | Pacotes

Quando estamos programando em Java, é comum criarmos muitas classes. Algumas podem estar relacionadas entre si, outras podem ter funções diferentes. Por exemplo, poderíamos ter classes relacionadas à interface do usuário em um grupo, classes de banco de dados em outro e assim por diante.

Para evitar que todas essas classes fiquem misturadas em um grande "bolo", o que tornaria difícil encontrá-las e usá-las depois, nós as agrupamos em **packages**, ou **pacotes**.

Lembra daquele exemplo da Netflix? Poderíamos ter um pacote chamado **com.netflix.interfaceUsuario** para todas as classes relacionadas à interface do usuário e **com.netflix.bancoDeDados** para todas as classes que interagem com o banco de dados.

Desta forma, quando queremos usar uma classe, sabemos exatamente onde procurá-la porque sabemos qual pacote ela está. É como se fôssemos diretamente para a gaveta certa no armário para pegarmos o precisamos!

Além disso, os packages também nos ajudam a evitar conflitos de nomes de classes. Sabemos que não podemos ter duas classes com o mesmo nome no mesmo pacote, mas podemos ter classes com o mesmo nome em pacotes diferentes. Isso é muito útil quando estamos trabalhando em projetos grandes, em que podemos precisar de classes com nomes semelhantes para funções diferentes.

---

**Resumo:** pacotes (*packages*) no Java são como gavetas organizadoras para as nossas classes, e que nos ajudam a encontrar, usar e

controlar o acesso às classes de uma forma mais organizada.

Finalmente, os packages também nos ajudam a controlar o acesso. As classes em um pacote podem acessar outras classes e membros no mesmo pacote que são declarados sem um modificador de acesso específico (como **public** ou **private**), o que nos dá mais controle sobre quem pode fazer o que com nossas classes. Já iremos mostrar como funcionam os packages, mas, primeiramente, precisamos falar sobre esses modificadores de acesso.

## | Modificadores de acesso: público, privado e pacote

Você já deve ter visto casualmente nos exemplos das semanas anteriores algumas menções a palavras como **public** e **private**, não? O Java utiliza algumas palavras-chave para ocultar os detalhes de implementação das classes, que são os modificadores de acesso. Eles são utilizados para especificar quais membros da classe (atributos ou métodos) podem ser visíveis ou acessíveis por outras classes.

O que acha de testarmos isso?



### EXEMPLO

**Exemplo de aplicação 1:** Crie um algoritmo em Java contendo uma classe chamada **Automovel**. Ela deve possuir atributos públicos (*public*), protegidos (*protected*) e privados (*private*).

Para resolver esse exemplo, crie um projeto. Esse novo projeto precisa de dois arquivos: **Teste.java** e **Automovel.java**.

Dentro do arquivo **Automovel.java**, cole o código a seguir:

```
</>
```

```
1 public class Automovel {
2     public String marca;
3     protected String modelo;
4     private String placa;
5
6     public Automovel(String marca, String modelo, String placa) {
7         this.marca = marca;
8         this.modelo = modelo;
9         this.placa = placa;
10    }
11
12    public String getPlaca() {
13        return this.placa;
14    }
15 }
```

Dentro do arquivo **Teste.java**, cole o arquivo a seguir:

</>



```
1 public class Teste {
2     public static void main(String[] args) {
3         Automovel auto = new Automovel("Chevrolet", "Onix", "ABC1234");
4
5         System.out.println("Marca: " + auto.marca); // Acessível porque é public
6         System.out.println("Modelo: " + auto.modelo); // Acessível porque é protected
7         // System.out.println("Placa: " + auto.placa); // Erro! Não temos acesso ao atributo placa diretamente
8         System.out.println("Placa: " + auto.getPlaca()); // Acessível através do método público getPlaca
9     }
10 }
```

Ao executar o código em **Teste.java** você verá que os detalhes do Chevrolet Onix aparecerão na tela. Agora, se você remover as duas barras no início da linha 7, se deparará com um erro do tipo “**java: placa has private access in Automovel**”. Esse é um erro de acesso proibido à placa. Afinal de contas, na linha 4 do **Automovel.java**, você pode ver que ela está marcada como **private** (privada). Em outras palavras, outras classes não poderão acessar diretamente aquele valor.

---

O **conjunto de métodos** de uma classe especificados como acessíveis por outras classes constitui a **interface** dessa classe.

Vamos entender um pouco mais sobre a diferença entre *public*, *private* e *protected*? Na Figura a seguir estão apresentados os modificadores de acesso público, privado e protegido. O modificador protegido será abordado novamente quando estivermos estudando a **Unidade 5 – Hierarquia**.

---

Modificador **público** em Java = `public`:

Indica que a classe, atributo ou método é visível por qualquer outra classe, em qualquer pacote.

Modificador **default** (padrão) em Java = não usa palavra-chave; vazio:

Indica que a classe, atributo ou método é apenas por classes que estejam no mesmo pacote ( `package` ) da classe em que foram declarados.

Modificador **protegido** em Java = `protected` :

Indica que o atributo ou método é visível apenas por classes que estejam no mesmo pacote ( `package` ) da classe em que foram declarados, ou em classes filhas (subclasse) de qualquer pacote\*.

Modificador **privado** em Java = `private` :

Indica que o atributo ou método é visível apenas dentro da própria classe em que foi declarado.

| Visibilidade                       |              |            |                |       |
|------------------------------------|--------------|------------|----------------|-------|
| Modificador em Java                | <i>Class</i> | Subclasse* | <i>Package</i> | Mundo |
| Sem modificador ( <i>default</i> ) | ✓            | ✗          | ✓              | ✗     |
| <i>public</i>                      | ✓            | ✓          | ✓              | ✓     |
| <i>protected</i> *                 | ✓            | ✓          | ✓              | ✗     |

|                |   |   |   |   |
|----------------|---|---|---|---|
| <i>private</i> | ✓ | × | × | × |
|----------------|---|---|---|---|

\*Veremos detalhes do conceito de **classes filha** (subclasses) com o modificador **protected** na Unidade 5 – Hierarquia.

Para exemplificar, vamos pensar em uma solução computacional que é concebida a partir de um exercício de abstração, em que precisamos identificar os elementos que interessam a um determinado software. Nos exemplos a seguir, observe **como o código é feito**. Veja e compare as diferenças na sintaxe e no uso de membros públicos, privados e protegidos (de pacote).

## Membros públicos

---

Como vimos, um membro (atributo ou método) público é **visível e acessível por qualquer outra classe**, logo:

- Um método público pode ser **chamado** de qualquer classe.
- Um atributo público pode ser **lido** ou **escrito** a partir de qualquer classe.

```
class A {
    public static void g( ) // método A.g() é público
    { ... }
    public static int k;    // atributo A.k é público
}
```

```
class B {
    static void f( ) // método B.f() é default
    {
```

```
    A.g();        // B.f() chama o método A.g()
    int x = A.k;   // B.f() lê o atributo A.k
    A.k = 10;      // B.f() escreve ou modifica o atributo A.k
}
}
```



### IMPORTANTE

Veja que o método `f` da classe `B` não possui nada antes do “*static void*”. Quer dizer, não tem nem “*public*”, nem “*private*”, nem “*protected*”. Esses casos entram na regra de acesso padrão (*default*): ele é mais restrito do que um “*public*”, mas menos restrito do que um “*private*”.

## Membros privados

Como vimos, um membro (atributo ou método) privado **tem o seu acesso restrito por outras classes**. Logo:

- Um método privado pode ser **chamado apenas dentro na própria classe** em que foi declarado.
- Um atributo privado pode ser **lido ou escrito apenas dentro na própria classe** em que foi declarado.

```
class A {
    private static void g( ) // método A.g() é privado
    { ... }
    private static int k;    // atributo A.k é privado

    static void f( )         // método A.f() é default
    {
        g();                // método A.g() é chamado na própria classe, em A.f()
    }
}
```

```
        int x = k;  
        k = 10;    // atributo A.k é lido na própria classe, em A.f()  
    }  
}
```

## Membros de pacote

---

Lembre-se de que usamos os pacotes ( `packages` ) para **agrupar classes**.

- Um **membro** (atributo ou método) de uma **classe declarada dentro de pacote** é **visível e acessível** por todas as **classes desse mesmo pacote** e somente por essas.
- Importante: quando o **modificador não é especificado**, a classe pertence ao pacote ***default*** (padrão).

Classe **A** e **B** pertencem ao pacote **P1**

```
package P1;  
class A{  
    ...  
}
```

```
package P1;  
class B{  
    ...  
}
```

Classe **C** e **D** pertencem ao pacote **P2**

```
package P2;  
class C{  
...  
}
```

```
package P2;  
class D{  
...  
}
```

Classe **E** e **F** pertencem ao pacote ***default***

```
// default  
class E{  
...  
}
```

```
// default  
class F{  
...  
}
```



## DICA

Para usar o **package** em seu código é simples: na primeira linha do arquivo Java basta colocar a palavra-chave “**package**”, seguida pelo nome que deseja atribuir ao seu pacote. Lembre-se que os pacotes devem ser usados para agrupar diferentes classes dentro de um mesmo projeto.

## Classes públicas

Uma classe pública é visível e acessível a partir de **classes de outros pacotes**.

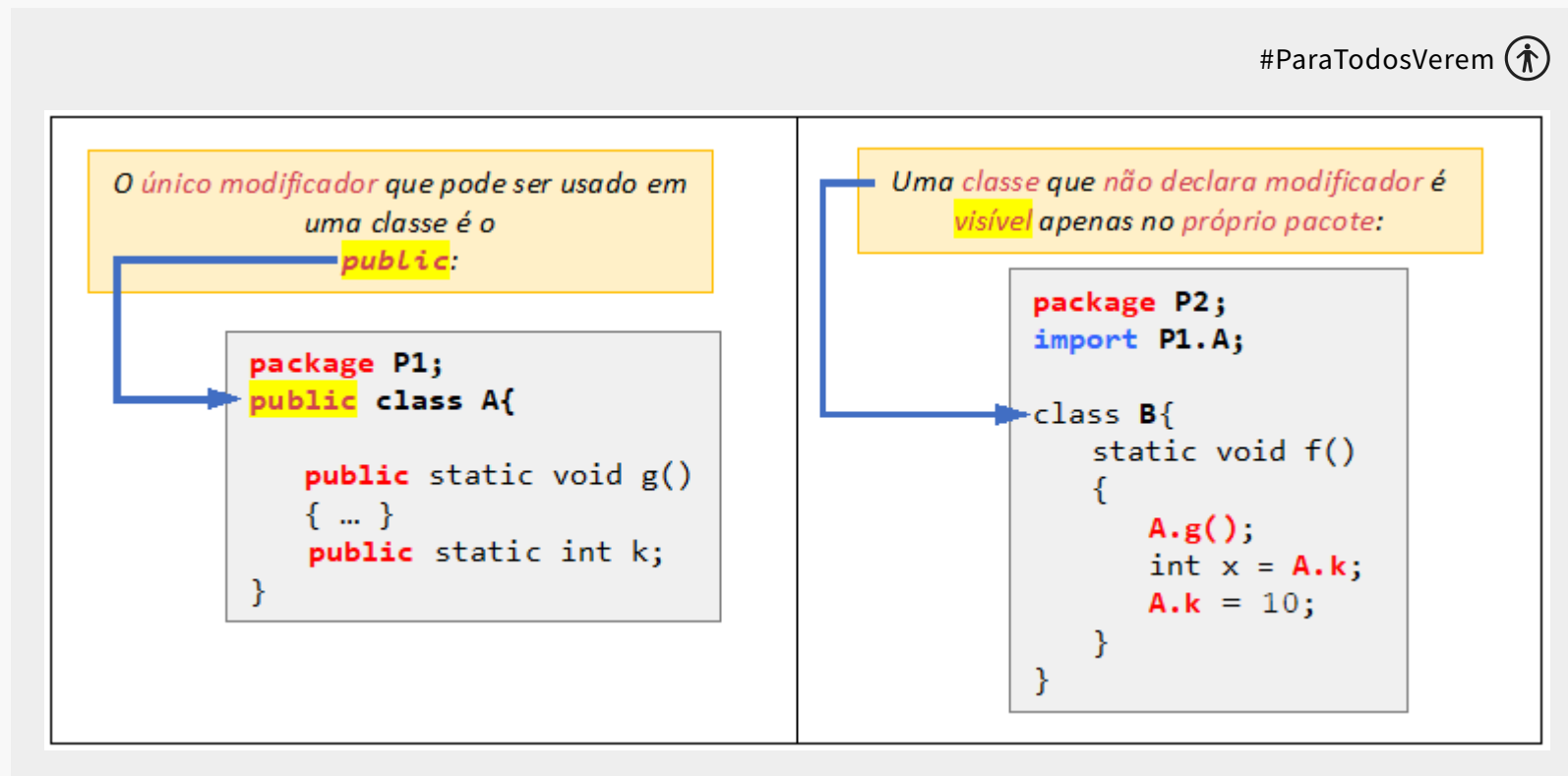


Figura 2: Exemplo de classes públicas. Fonte: Autores (2021)

O que acha de fazermos mais um exercício?

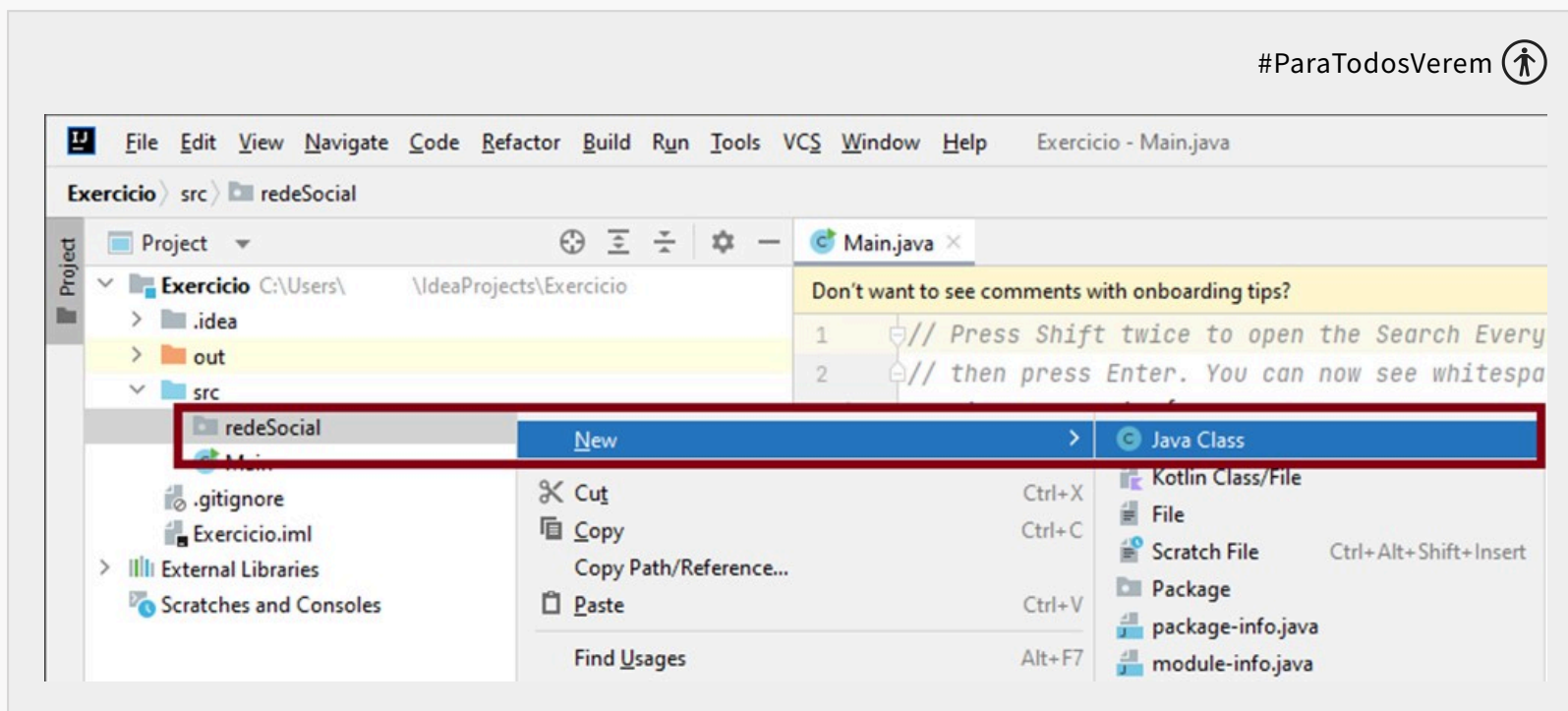


## EXEMPLO

**Exemplo de aplicação 2:** crie um projeto em Java de uma rede social contendo duas classes: **Postagem** e **Usuario**. Ambas as classes pertencem ao pacote **redeSocial**. Não use modificadores de acesso.

Crie um pacote em seu projeto. Se estiver usando o IntelliJ, clique em **File > New > Package**. O nome do pacote deve ser **redeSocial**. Aparecerá no menu lateral um novo pacote com o nome de **redeSocial**, e com um desenho parecido com o de uma pasta.

Clique com o botão direito neste pacote, e escolha **New > Java Class**, seguindo o exemplo mostrado a seguir.





Crie um arquivo chamado **Postagem.java**. Repita o mesmo processo – agora, para criar **Usuario.java**.

Copie e cole o código a seguir para o arquivo **Postagem.java**:

</>

```
1 package redeSocial;
2
3 public class Postagem {
4     int curtidas;
5
6     static void mostrarCurtidas() {
7         System.out.println("-----");
8         System.out.println("... Postagem recebeu curtida!");
9         System.out.println("... Total de curtidas = " + curtidas);
10        System.out.println("-----");
11    }
12 }
```

Copie e cole o código a seguir para o arquivo **Usuario.java**:

</>

```
1 package redeSocial;
2
3 public class Usuario {
4     void curtirPostagem(Postagem p) {
5         System.out.println("Usuário curtiu a postagem");
6
7         int x = p.curtidas;
8         System.out.println("Curtidas antes = " + x);
9         p.curtidas++;
10        System.out.println("Curtidas agora = " + p.curtidas);
11    }
12
13    public static void main(String args[]) {
14        Usuario user = new Usuario();
15        Postagem post = new Postagem();
16        user.curtirPostagem(post);
17    }
18 }
```

Agora, execute o seu código a partir do **Usuario.java**. Você verá isto como resultado:

Usuário curtiu a postagem

-----

... Postagem recebeu curtida!

... Total de curtidas = 0

-----

Curtidas antes = 0

Curtidas agora = 1

Beleza, funcionou. Agora, vamos entender melhor o que acabamos de fazer?

1. **Postagem.java** (linhas 4 e 6) e **Usuario.java** (linha 4): não declaramos modificadores de acesso. Logo, o modificador é default (padrão vazio).
2. **Postagem.java** (linha 4): observe que o atributo `Postagem.curtidas` não tem valor declarado e, nesse caso, seu valor default (valor-padrão) é zero.
3. **Postagem.java** (linha 6): observe que o método `Postagem.mostrarCurtidas()` apenas apresenta o valor de `Postagem.curtidas` (linha 4) na tela.
4. **Usuario.java** (linhas 7, 9 e 10): observe que a classe `Usuario` tem acesso aos membros da classe `Postagem`, pois o modificador dos seus membros é o default e ambas as classes estão no mesmo pacote `redeSocial`.
5. **Usuario.java** (linhas 4-11): Observe que o método `Usuario.curtirPostagem()` (linha 4) consegue alterar o atributo `Postagem.curtidas` – a classe **Usuario** tem permissão para alterar as coisas da classe **Postagem** porque o modificador de `Postagem.curtidas` é o default.

Agora, verifique as questões a seguir no seu ambiente de desenvolvimento (IDE) e responda:



## EXERCÍCIO

1. É possível usar o modificador `private` antes da palavra-chave `class`, tanto na classe A quanto na classe B? O que acontece?
2. É possível usar o modificador `private` nos métodos `Postagem.mostrarCurtidas()` e `Usuario.curtirPostagem()`? Justifique.
3. É possível usar o modificador `protected` nos métodos `Postagem.mostrarCurtidas()` e `Usuario.curtirPostagem()`? Justifique.



## Resolução do exercício



1. Não é possível. Gera erro de compilação, pois a classe precisa ser pública ou default (vazio).

2. O método `Postagem.mostrarCurtidas()` não pode ter modificador `private` porque ele é utilizado nos objetos da classe `Usuario`. O método `Usuario.curtirPostagem()` poderia ter modificador `private` porque apenas é utilizado dentro da própria classe `Usuario`.
  3. Poderíamos utilizar o modificador `protected` em ambos os métodos `Postagem.mostrarCurtidas()` e `Usuario.curtirPostagem()`, pois são classes que estão no mesmo pacote e, com esse modificador, os métodos estariam acessíveis entre as classes.
- 

## | Métodos `getter` e `setter`

Vimos anteriormente que podemos **restringir o acesso** a determinados métodos ou atributos com o uso de modificadores de acesso. Vamos supor, por exemplo, de que você possui alguns atributos configurados como **`private`**. Por outro lado, não gostaria de proibir o acesso para todos: talvez você só tivesse a intenção de **restringir e controlar** o acesso para estes atributos.

Imaginemos o caso da Netflix: você poderia deixar o atributo correspondente às séries assistidas por um usuário como **`private`**. Isso significa que essa é uma informação proibida e supersecreta? Não exatamente: às vezes, poderíamos deixá-la como **`private`** porque queremos filtrar, organizar ou tratar aquela informação antes de permitir que outras classes acessem aquele valor.

Assim, quando o encapsulamento oculta atributos da classe, usando o modificador de acesso `private`, é possível permitir o acesso a esses atributos de uma maneira controlada. A prática mais comum para fazer isso é criar dois métodos-padrão `getter` (que **obtem** ou **retorna** alguma coisa) e `setter` (que **atribui** ou **modifica** alguma coisa). Esses métodos compõem a interface da classe e visam a prover um acesso controlado a atributos encapsulados. Veja como são declarados:

1. Padrão do método **`getter`** – retorna o valor do atributo: `getNomeAtributo`.
2. Padrão do método **`setter`** – muda o valor do atributo: `setNomeAtributo`.
3. Padrão do método **`is`** – retorna o valor *boolean* do atributo: `isAtributo`.

Verifique e execute os exemplos de *getter* e *setter* nas Figuras 9 e 10 a seguir.



### EXEMPLO

**Exemplo de aplicação 3:** crie um projeto em Java contendo uma classe chamada **Carro**. Ela possui dois atributos privados: **marca** (o nome da marca) e **ligado** (se o carro está ligado ou desligado). A classe precisa de métodos públicos *getter* e *setter* para cada um destes atributos.

Logo, cole e teste o código a seguir em seu arquivo **Carro.java**:

</>

```
1 public class Carro {
2     private String  marca; // atributo oculto (encapsulado/privado)
3     private boolean ligado; // atributo oculto (encapsulado/privado)
4
5     public Carro(String marca, boolean ligado) { // Método construtor
6         this.marca = marca;
7         this.ligado = ligado;
8     }
9
10    public String getMarca() { // Retorna valor do atributo privado "marca";
11        return marca;
12    }
13
14    public void setMarca(String marca) { // Altera valor de "marca";
15        this.marca = marca;
16    }
17
18    public boolean isLigado() { // Retorna valor do atributo privado "ligado";
19        return ligado;
20    }
21 }
```

Vamos entender este código?

1. **Linhas 2 e 3:** Aqui, estamos declarando dois atributos privados da classe **Carro**: a **marca** (uma String que indica a marca do carro) e **ligado** (um booleano que indica se o carro está ligado ou não).
2. **Linhas 5-8:** Aqui temos a definição do construtor **public Carro(String marca, boolean ligado)**. Este método é chamado quando criamos um objeto da classe **Carro**. Ele recebe a marca do carro e um valor booleano indicando se o carro está ligado ou não.
3. **Linhas 10-24:** Definimos os getters e setters dos atributos privados com os métodos **getMarca()**, **setMarca(String marca)**, **isLigado()** e **setLigado(boolean ligado)**. Os métodos getter retornam o valor atual do atributo, enquanto os métodos setter permitem modificar o valor do atributo. Nem todo atributo privado precisa de getters e setters – os criamos conforme a necessidade.

1. `public String getMarca()` – retorna valor do atributo privado “marca”.
  2. `public void setMarca(String marca)` – altera valor do atributo privado “marca”.
  3. `public boolean isLigado()` – retorna valor do atributo privado “ligado”.
  4. `public void setLigado(boolean ligado)` – altera valor do atributo privado “ligado”.
4. **Linhas 26-36:** O método `main(String[] args)`: Este é o ponto de entrada do programa. Ele cria um objeto da classe **Carro** chamado **meuCarro**, inicialmente com a marca "Ford" e o estado **ligado** como **false**. Em seguida, ele imprime a marca do carro e se ele está ligado ou não.
5. **Linha 29:** Percebeu algo de diferente aqui? Este é um **operador ternário**. O operador ternário (`meuCarro.isLigado() == true ? "Ligado" : "Desligado"`) é um atalho para um comando if-else. Ele verifica se `meuCarro.isLigado()` é **true**. Se for, ele retorna "Ligado". Caso contrário, retorna "Desligado".



#### DICA

A sintaxe de um operador ternário é sempre (**condição ? resultado\_se\_verdadeiro : resultado\_se\_falso** ). Se você trabalha com Excel, ele possui um funcionamento parecido com a função **=SE(condição;[resultado\_se\_verdadeiro];[resultado\_se\_falso])**.

1. **Linhas 31 e 32:** Estas linhas estão mudando os atributos do objeto **meuCarro** usando os métodos setters. A marca está sendo alterada para "Renault" e o estado **ligado** para **true**.
2. **Linhas 34 e 35:** Por fim, ele imprime novamente a marca do carro e se ele está ligado ou não, desta vez refletindo as mudanças que foram feitas.

O que acha de outro exemplo?



#### EXEMPLO



**Exemplo de aplicação 4:** crie um projeto em Java contendo um arquivo chamado **Conta.java**, contendo o código a seguir.

</>

```
1 public class Conta {
2     private double saldo;
3     private String dono;
4
5     public String getDono() {
6         return dono;
7     }
8
9     public void setDono(String dono) {
10        this.dono = dono;
11    }
12
13    public double getSaldo() {
14        return saldo;
15    }
16
17    public void setSaldo(double saldo) {
18        this.saldo = saldo;
19    }
```

Agora, crie um arquivo com o nome **Banco.java**, contendo o código a seguir.

</>



```
1 public class Banco {
2     public static void main(String[] args) {
3         // Cria objetos da classe Conta
4         Conta cta1 = new Conta();
5         Conta cta2 = new Conta();
6
7         //código para alterar o saldo da cta1
8         //código para alterar o nome do dono da cta1
9
10        //código para alterar o saldo da cta2
11        //código para alterar o nome do dono da cta2
12
13        System.out.println("Cta 1 - Correntista: " + "XXXX");
14        System.out.println("Cta 1 - Saldo: " + "XXXX");
15
16        System.out.println("Cta 2 - Correntista: " + "XXXX");
17        System.out.println("Cta 2 - Saldo: " + "XXXX");
18    }
19 }
```

Agora, modifique as linhas 7-11 do código em **Banco.java** seguindo as instruções dessas linhas. Depois, substitua as Strings “XXXX” nas linhas 13-17 pelo saldo e nome corretos de cada uma das contas.

E agora, como resolveríamos isso? O segredo é olharmos como as coisas estão na classe **Conta**. Veja que já temos o nome dos setters (**setDono()** e **setSaldo()**). Logo, vamos alterar as linhas 7-11 para incluir estes dois setters corretamente. O código ajustado do **Banco.java** ficará assim:

</>

```
1 public class Banco {
2     public static void main(String[] args) {
3         // Cria objetos da classe Conta
4         Conta cta1 = new Conta();
5         Conta cta2 = new Conta();
6
7         cta1.setSaldo(1000.50);
8         cta1.setDono("Fulano de Tal");
9
10        cta2.setSaldo(5555.99);
11        cta2.setDono("Beltrano da Silva");
12
13        System.out.println("Cta 1 - Correntista: " + "XXXX");
14        System.out.println("Cta 1 - Saldo: " + "XXXX");
15
16        System.out.println("Cta 2 - Correntista: " + "XXXX");
17        System.out.println("Cta 2 - Saldo: " + "XXXX");
18    }
19 }
```

Perceba as linhas 7 a 11, que foram modificadas. Veja como estamos usando os setters: **setSaldo()** espera um número, e é isto que estamos informando. Já **setDono()** espera uma String, e também estamos informando isto.

Agora, precisamos mostrar estas informações corretamente nas linhas 13-17 ao substituir o “XXXX”. É para isto que servem os getters. Veja como ficará o código modificado usando os métodos **getDono()** e **getSaldo()**:

</>

```
1 public class Banco {
2     public static void main(String[] args) {
3         // Cria objetos da classe Conta
4         Conta cta1 = new Conta();
5         Conta cta2 = new Conta();
6
7         cta1.setSaldo(1000.50);
8         cta1.setDono("Fulano de Tal");
9
10        cta2.setSaldo(5555.99);
11        cta2.setDono("Beltrano da Silva");
12
13        System.out.println("Cta 1 - Correntista: " + cta1.getDono());
14        System.out.println("Cta 1 - Saldo: " + cta1.getSaldo());
15
16        System.out.println("Cta 2 - Correntista: " + cta2.getDono());
17        System.out.println("Cta 2 - Saldo: " + cta2.getSaldo());
18    }
19 }
```



### EXEMPLO

**Exemplo de aplicação 5:** crie um projeto em Java contendo dois arquivos – **Arquiteto.java** e **Circulo.java**.

Cole o código a seguir dentro do arquivo **Arquiteto.java**.

</>

```
1 public class Arquiteto {
2     private String nome;
3     private int idade;
4
5     // Método construtor (tem o mesmo nome da classe)
6     public Arquiteto(String nome, int idade) {
7         this.nome = nome;    // "this" = referência ao atual objeto
8         this.idade = idade;
9     }
10
11     public void exibirDadosPessoais(){
12         System.out.println(nome);
13         System.out.println(idade+ " anos");
14     }
15
16     public void trabalhe (double r1, double r2, double r3) {
17         Circulo a = new Circulo(r1); // define e instancia o objeto "a"
18         double x = a.area();
19         double y = a.perimetro();
```

Cole o código a seguir dentro do arquivo **Circulo.java**.

</>

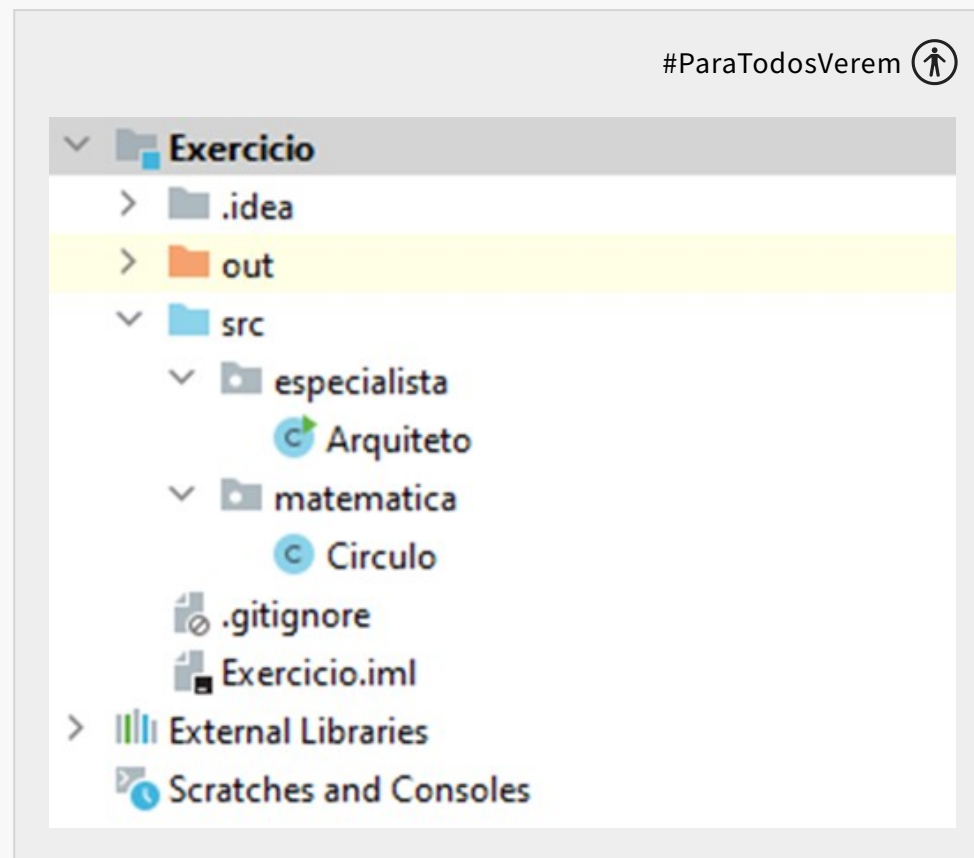
```
1 public class Circulo {
2     private static double PI = 3.141516;
3     private double raio;
4
5     public Circulo(double raio) {
6         this.raio = raio;
7     }
8
9     public double area(){
10         return PI * raio * raio ;
11     }
12
13     public double perimetro (){
14         return 2 * PI * raio;
15     }
16 }
```

Depois de testar o código você verá que aparecerão as informações de três círculos (raio, área e perímetro dos três círculos) na tela. Perceba o seguinte:

1. Veja em quais linhas e em quais casos o public e private são usados nos dois arquivos.
2. No arquivo **Arquiteto.java** veja três formas diferentes de instanciar a classe Circulo:
  1. Linhas 17, 18 e 19: informamos o tipo de dado das variáveis **a**, **x** e **y**.
  2. Linhas 23 e 24: aqui, não precisamos mais definir o tipo de dado das variáveis **x** e **y**. Afinal, elas já existem e já sabemos que são do tipo **double**. Aqui, só estamos reutilizando (e sobrescrevendo) o valor destas variáveis.
  3. Linha 27: aqui, também não precisamos mais definir o tipo da variável **b**, já que ela foi definida na linha 22. Aqui, só estamos reutilizando (e sobrescrevendo) o valor desta variável.
3. Remova os modificadores public dos membros da classe Circulo (linhas 5, 9 e 13 do arquivo **Circulo.java**) e repita o teste. Por que ainda funciona?

1. Resposta: porque um atributo ou método default (identificados pela **ausência** de modificador) pode ser acessado por objetos de todas as classes que estiverem no mesmo pacote que a classe que possui o atributo ou o método. Nesse exemplo, **Arquiteto** e **Circulo** estão no mesmo pacote.
2. **Ah, aviso importante:** Ao mover para pacotes diferentes, o seu código vai apresentar erros vermelhos imediatamente. Não se preocupe, isto é esperado e vamos corrigir no próximo passo.

Agora, crie dois pacotes: **matematica** e **especialista**. Mova o arquivo **Circulo** para o pacote **matematica**, e o arquivo **Arquiteto** para o pacote **especialista**. O seu projeto deverá ficar com uma estrutura de código semelhante a esta:



Fonte: Autores (2021).

A primeira linha do arquivo **Circulo.java** agora deverá conter o código `package matematica;` (sem aspas) em sua primeira linha.

Já o arquivo **Arquiteto.java** agora deverá conter o código “`package especialista;`” (sem aspas) na primeira linha, e “`import matematica.Circulo;`” na segunda linha. Sem isso, a classe **Arquiteto** não conseguirá “enxergar” a classe **Circulo**.

Ao tentar executar o projeto a partir do arquivo **Arquiteto.java**, você perceberá que haverá vários erros no seu código. Algo como “*area() is not public in matematica.Circulo; cannot be accessed from outside package*”. Erros assim implicam que estes métodos precisam estar como **public** para que possam ser acessados por classes em outros pacotes. Por isso, altere novamente os métodos **area()** e **perimetro()** no arquivo **Circulo.java** para que voltem a ser **public**, e teste novamente. Somente assim a classe **Arquiteto** (que está em outro pacote) poderá acessar os seus dados.

## | Classe String

Lá em Python (e em Raciocínio Computacional), aprendemos que as strings são uma **sequência de caracteres**. Java também possui Strings. Todos os tipos de dados utilizados no Java, com exceção dos tipos primitivos (int, double, char e boolean), são objetos. Logo, uma string, contida na biblioteca padrão **java.lang**, também é um objeto e deve ser **declarada** e **instanciada**. Ou seja: em Java, **strings também são classes**.

Pratique a criação e manipulação de strings nos exercícios a seguir.



#### EXEMPLO

**Exemplo de aplicação 6:** crie um projeto em Java contendo um arquivo chamado **StringDemo.java**. Cole o código a seguir dentro desse arquivo.

```
</>
```



```
1 public class StringDemo {
2
3     public static void main(String[] args) {
4         String ola = "Olá"; // Declara e instancia diretamente
5
6         String nome1 = new String("Maria"); // Instancia com new
7         String nome2 = new String("João"); // Instancia com new
8
9         // Declara e instancia com a concatenação de strings
10        String saudacao = ola + ", " + nome1 + "!\n\n";
11        System.out.println(saudacao);
12
13        // Altera o valor de saudacao pela atribuição do valor de ola
14        // Não é preciso declarar saudacao novamente
15        saudacao = ola;
16
17        // Altera o valor de saudacao, que recebe
18        // o resultado da concatenacao de mais strings
19        // É uma alternativa que utiliza o método
```

Aqui, mostramos algumas formas de usar e concatenar (juntar) Strings. Vejamos:

1. **String ola = "Olá";**: A string "Olá" é atribuída à variável **ola**.
2. **String nome1 = new String("Maria");** e **String nome2 = new String("João");**: Aqui estão sendo criados dois novos objetos de string, **nome1** e **nome2**, utilizando a palavra-chave **new**. Esses objetos recebem os valores "Maria" e "João", respectivamente.
3. **String saudacao = ola + ", " + nome1 + "!\n\n";**: aqui está sendo feita a concatenação de Strings. A variável **saudacao** é criada e recebe a combinação das Strings **ola**, **", "**, **nome1** e **!\n\n**, resultando na String "Olá, Maria!\n\n". A String "\n\n" representa duas quebras de linha.
4. **System.out.println(saudacao);**: esta linha imprime o valor da String **saudacao**.
5. **saudacao = ola;**: nesta linha, o valor da variável **saudacao** é alterado para o valor da String **ola**, ou seja, "Olá". Isso significa que agora estas duas variáveis possuem o mesmo valor.

6. `saudacao = saudacao.concat(", " + nome2);` e `saudacao = saudacao.concat("!\\n\\n");`: aqui é demonstrada outra maneira de concatenar Strings, usando o método `concat()`. A String ", João" é adicionada a `saudacao`, resultando em "Olá, João". Em seguida, a String "\\n\\n" é adicionada a `saudacao`, resultando em "Olá, João!\\n\\n".
7. `System.out.println(saudacao + "fim");`: finalmente, a String "fim" é adicionada à `saudacao` usando o operador `+` e a String resultante é impressa, resultando em "Olá, João!\\n\\nfim".



### EXEMPLO

**Exemplo de aplicação 7:** Crie um projeto em Java contendo um arquivo chamado **StringExamples.java**. Cole o código a seguir dentro deste arquivo.

</>

```

1 public class StringExamples {
2
3     public static void main(String[] args) {
4         String txt;
5
6         txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
7         System.out.println("Comprimento de [" + txt + "] = " + txt.length());
8
9         txt = "Olá Mundo!";
10        System.out.println(txt.toUpperCase());    // Saída = "OLÁ MUNDO!"
11        System.out.println(txt.toLowerCase());    // Saída = "olá mundo"
12
13        txt = "Localize a posição da palavra 'eureka' neste texto!";
14        System.out.println(txt.indexOf("eureka")); // Saída = 31
15    }
16 }

```

Veja que este código demonstra algumas das operações comuns que podem ser realizadas em objetos String. Vamos passar por cada seção:

1. **String txt;**: aqui, estamos declarando uma variável chamada **txt** do tipo **String**. Ela será usada para armazenar várias Strings ao longo do programa.
2. **txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";**: nesta linha, atribuímos à variável **txt** a String contendo todas as letras do alfabeto em maiúsculas.
3. **System.out.println("Comprimento de [" + txt + "] = " + txt.length());**: aqui, estamos imprimindo o comprimento da string armazenada em **txt** usando o método **length()**. Portanto, a saída será "Comprimento de [ABCDEFGHIJKLMNOPQRSTUVWXYZ] = 26".
4. **txt = "Olá Mundo!";**: agora, estamos alterando o valor de **txt** para "Olá Mundo!".
5. **System.out.println(txt.toUpperCase());** e **System.out.println(txt.toLowerCase());**: Nestas linhas, estamos demonstrando o uso dos métodos **toUpperCase()** e **toLowerCase()**, que transformam todas as letras da string em maiúsculas e minúsculas, respectivamente. Portanto, as saídas serão "OLÁ MUNDO!" e "olá mundo".

6. `txt = "Localize a posição da palavra 'eureka' neste texto!";` Aqui, alteramos o valor de `txt` para a frase especificada.
7. `System.out.println(txt.indexOf("eureka"));` Finalmente, demonstramos o uso do método `indexOf()`, que retorna a posição da primeira ocorrência da substring especificada na String. Nesse caso, estamos procurando a posição de "eureka" na frase, que começa no índice 31 (lembrando que a contagem começa de 0). Portanto, a saída será 31. A lógica aqui é a mesma ideia das listas que trabalhamos em Python, anteriormente.

## | Métodos de String

A tabela a seguir apresenta alguns dos métodos da classe `java.lang.String`. A lista completa, com exemplos de utilização, pode ser encontrada em ORACLE, 2023 e em W3SCHOOLS, 2023.

| Método                                              | Descrição                                                                                                                         | Tipo de retorno |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <code>charAt()</code>                               | Retorna o caractere no índice especificado (posição).                                                                             | <i>char</i>     |
| <code>concat(String str)</code>                     | Concatena a <i>string</i> especificada ao final desta <i>string</i> .                                                             | <i>string</i>   |
| <code>equals(Object anObject)</code>                | Compara esta <i>string</i> com o objeto especificado.                                                                             | <i>boolean</i>  |
| <code>equalsIgnoreCase(String anotherString)</code> | Compara esta <i>string</i> com outra <i>string</i> , ignorando diferenças de maiúsculas e minúsculas ( <i>case insensitive</i> ). | <i>boolean</i>  |
| <code>indexOf(int ch)</code>                        | Retorna o índice dentro desta <i>string</i> da primeira ocorrência do caractere especificado.                                     | <i>int</i>      |
| <code>indexOf(String str)</code>                    | Retorna o índice dentro desta <i>string</i> da primeira ocorrência da <i>substring</i> especificada.                              | <i>int</i>      |
| <code>isEmpty()</code>                              | Retorna verdadeiro se, e somente se, <i>length()</i> for 0.                                                                       | <i>boolean</i>  |
| <code>length()</code>                               | Retorna o comprimento desta <i>string</i> .                                                                                       | <i>int</i>      |

|                                             |                                                                                                                                              |               |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>replace</b> (char oldChar, char newChar) | Retorna uma nova <i>string</i> resultante da substituição de todas as ocorrências de <i>oldChar</i> nesta <i>string</i> por <i>newChar</i> . | <i>string</i> |
| <b>substring</b> (int beginIndex)           | Retorna uma nova <i>string</i> que é uma <i>substring</i> desta <i>string</i> .                                                              | <i>string</i> |
| <b>toLowerCase</b> ()                       | Converte todos os caracteres desta <i>string</i> em minúsculas.                                                                              | <i>string</i> |
| <b>toUpperCase</b> ()                       | Converte todos os caracteres desta <i>string</i> em maiúsculas.                                                                              | <i>string</i> |
| <b>valueOf</b> (float f)                    | Retorna a representação de <i>string</i> do argumento <i>float</i> .                                                                         | <i>string</i> |
| <b>valueOf</b> (int i)                      | Retorna a representação de <i>string</i> do argumento <i>int</i> .                                                                           | <i>string</i> |

## Exercícios de fixação



### EXERCÍCIO

1. Crie uma classe "ContaBancaria" com atributos privados para "numeroDaConta", "agencia", "saldo" e "senha". Use métodos getter para "agencia", "numeroDaConta" e "saldo", mas apenas um método setter para "senha".
2. Crie uma classe "CorridaUber" que tenha atributos privados para "localizacaoAtual", "destino" e "precoEstimado". Use métodos getter e setter para acessar e modificar esses atributos.
3. Crie uma classe "Pessoa" com o atributo privado "idade". No método "setIdade(int novaldade)", adicione uma lógica (if/else) que impeça que a idade seja negativa ou maior que 150. Se o valor for inválido, imprima um erro e não altere o atributo.
4. Crie um método que receba uma String e retorne a mesma String, mas com a primeira letra de cada palavra em maiúsculo (como um título de filme). Exemplo: ao informar a String "o senhor dos anéis", o método deve retornar "O Senhor Dos Anéis".
5. Crie uma classe "PostInstagram" com atributos privados para "imagemURL", "descricao" e "quantidadeLikes". Use métodos getter e setter para acessar e modificar esses atributos.

6. Crie um método que receba uma String e retorne essa mesma string, mas ao contrário. Por exemplo: ao informar a String “Valorant”, o método deve retornar “tnarolaV”.
7. Crie uma classe "UsuarioGoogle" com atributos privados para "email" e "senha". Use métodos getter e setter para acessar e modificar esses atributos.
8. Crie uma classe "VagaLinkedIn" com atributos privados para "empresa", "posicao" e "descricao". Use métodos getter e setter para acessar e modificar esses atributos.



## Resolução do exercício



Exercício 1:

</>

```
1 public class ContaBancaria {
2     private String numeroDaConta;
3     private String agencia;
4     private double saldo;
5     private String senha;
6
7     public ContaBancaria(String numeroDaConta, String agencia, double saldo, String senha) {
8         this.numeroDaConta = numeroDaConta;
9         this.agencia = agencia;
10        this.saldo = saldo;
11        this.senha = senha;
12    }
13
14    public String getAgencia() {
15        return agencia;
16    }
17
18    public String getNumeroDaConta() {
19        return numeroDaConta;
```

Exercício 2:

</>

```
1 public class CorridaUber {
2     private String localizacaoAtual;
3     private String destino;
4     private double precoEstimado;
5
6     public CorridaUber(String localizacaoAtual, String destino, double precoEstimado) {
7         this.localizacaoAtual = localizacaoAtual;
8         this.destino = destino;
9         this.precoEstimado = precoEstimado;
10    }
11
12    public String getLocalizacaoAtual() {
13        return localizacaoAtual;
14    }
15
16    public void setLocalizacaoAtual(String localizacaoAtual) {
17        this.localizacaoAtual = localizacaoAtual;
18    }
19 }
```

Exercício 3:

</>



```
1 public class Pessoa {
2     // 1. Atributo privado: ninguém de fora pode mexer diretamente aqui
3     private int idade;
4
5     // 2. Método getter: permite apenas LER o valor
6     public int getIdade() {
7         return this.idade;
8     }
9
10    // 3. Método setter: é ele que valida a entrada
11    public void setIdade(int novaIdade) {
12        // Lógica de Proteção: Verifica se a idade é inválida
13        // (Menor que 0 OU Maior que 150)
14        if (novaIdade < 0 || novaIdade > 150) {
15            System.out.println("ERRO: A idade " + novaIdade + " é inválida! A alteração foi
16        } else {
17            // Se passou na validação, permitimos a gravação no atributo
18            this.idade = novaIdade;
19            System.out.println("Sucesso: Idade definida para " + this.idade);
```

Exercício 4:

</>

```
1 public class StringUtils {
2
3     public static String toTitleCase(String input) {
4         StringBuilder titleCase = new StringBuilder();
5         boolean nextTitleCase = true;
6
7         for (char c : input.toCharArray()) {
8             if (Character.isSpaceChar(c)) {
9                 nextTitleCase = true;
10            } else if (nextTitleCase) {
11                c = Character.toTitleCase(c);
12                nextTitleCase = false;
13            }
14
15            titleCase.append(c);
16        }
17
18        return titleCase.toString();
19    }
```

Exercício 5:

</>

```
1 public class PostInstagram {
2     private String imagemURL;
3     private String descricao;
4     private int quantidadeLikes;
5
6     // Método construtor
7     public PostInstagram(String imagemURL, String descricao, int quantidadeLikes) {
8         this.imagemURL = imagemURL;
9         this.descricao = descricao;
10        this.quantidadeLikes = quantidadeLikes;
11    }
12
13    // Getters
14    public String getImagemURL() {
15        return this.imagemURL;
16    }
17
18    public String getDescricao() {
19        return this.descricao;
```

Exercício 6:

</>

```
1 public class StringInverter {
2
3     public static String invertString(String original) {
4         String invertida = "";
5         for(int i = original.length() - 1; i >= 0; i--) {
6             invertida = invertida + original.charAt(i);
7         }
8         return invertida;
9     }
10
11     public static void main(String[] args) {
12         String original = "Valorant";
13         String invertida = invertString(original);
14         System.out.println("String original: " + original);
15         System.out.println("String invertida: " + invertida);
16     }
17 }
```

Exercício 7:

</>

```
1 public class UsuarioGoogle {
2     private String email;
3     private String senha;
4
5     public UsuarioGoogle(String email, String senha) { // Construtor
6         this.email = email;
7         this.senha = senha;
8     }
9
10    public String getEmail() {
11        return email;
12    }
13
14    public void setEmail(String email) {
15        this.email = email;
16    }
17
18    public String getSenha() {
19        return senha;
```

Exercício 8:

</>

```
1 public class VagaLinkedIn {
2     private String empresa;
3     private String posicao;
4     private String descricao;
5
6     public VagaLinkedIn(String empresa, String posicao, String descricao) { // Construtor
7         this.empresa = empresa;
8         this.posicao = posicao;
9         this.descricao = descricao;
10    }
11
12    public String getEmpresa() {
13        return empresa;
14    }
15
16    public void setEmpresa(String empresa) {
17        this.empresa = empresa;
18    }
19 }
```

## | Referências Bibliográficas

GODOY, V. **Programação orientada a objetos I**. Curitiba, PR: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – Volume I**. 8. ed. São Paulo, SP: Pearson, 2010.

ORACLE. Class string. **Oracle**, 2023. Disponível em: <https://docs.oracle.com/javase/8/docs/api/java/lang/String.html>. Acesso em: 17 jul. 2023.

SCHILDT, H. **Java para iniciantes**. Porto Alegre, RS: Bookman, 2015.

W3SCHOOLS. **Java string methods**. **W3SCHOOLS**, 2021. Disponível em: [https://www.w3schools.com/java/java\\_ref\\_string.asp](https://www.w3schools.com/java/java_ref_string.asp). Acesso em: 17 jul. 2023.



© PUCPR - Todos os direitos reservados.